实验一

实验要求

使用 PyTorch 或 TensorFlow 手写一个前馈神经网络，用于近似函数：

$$y = \sin(x), \qquad x \in [0, 2\pi).$$

并研究网络深度、学习率、网络宽度、激活函数对模型性能的影响。

实验步骤

1. **网络框架：**要求选择 PyTorch 或 TensorFlow 其中之一，依据官方网站的指引安装包。若你需要使用 GPU，可能还需安装 CUDA 驱动。本次实验仅利用 CPU 也可以完成，但仍强烈推荐大家安装 GPU 版本，以满足后续实验需求。
2. **数据生成：**本次实验的数据集仅需使用程序自动生成，即在 $[0, 2\pi)$ 范围内随机 sample 样本作为 x 值，并计算 $y = \sin(x)$ 作为 y 值。要求生成三个**互不相交**的数据集分别作为训练集、验证集、测试集。训练只能在训练集上完成，实验调参只能在验证集上完成。
3. **模型搭建：**采用 PyTorch 或 TensorFlow 所封装的 module 编写模型，例如 `torch.nn.Linear()`、`torch.nn.ReLU()` 等，无需手动完成底层 forward、backward 过程。
4. **模型训练：**将生成的训练集输入搭建好的模型进行前向的 loss 计算和反向的梯度传播，从而训练模型，同时也建议使用网络框架封装的 optimizer 完成参数更新过程。训练过程中记录模型在训练集和验证集上的损失，并绘图可视化。
5. **调参分析：**将训练好的模型在验证集上进行测试，以 Mean Square Error（MSE）作为网络性能指标。然后，对网络深度、学习率、网络宽度、激活函数等模型超参数进行调整，再重新训练、测试，并分析对模型性能的影响。
6. **测试性能：**选择你认为最合适的（例如，在验证集上表现最好的）一组超参数，重新训练模型，并在测试集上测试（注意，这理应是你的实验中**唯一一次**在测试集上的测试），并记录测试的结果（MSE）。

实验提交

本次实验截止日期为 **11 月 14 日 23:59:59**，提交到邮箱 liangyunfei@mail.ustc.edu.cn，具体要求如下：

1. 全部文件打包在一个压缩包内，压缩包命名为 `学号-姓名-expl.zip`。
2. 代码仅包含 `.py` 文件，请勿包含实验中间结果（例如中间保存的数据集等），如果有多个文件，放在 `src/` 文件夹内。
3. 代码中提供一个可以直接运行的并输出结果的 `main.py`，结果包括训练集损失、验证集损失随 epoch 改变的曲线（保存下来）和测试集的 MSE。
4. 代码中提供一个描述所有需依赖包的 `requirements.txt`，手动列入代码中用到的所有非标准库及版本或者使用 `pip freeze > requirements.txt` 命令生成。
5. 实验报告要求 PDF 格式，要求包含姓名、学号。内容包括简要的实验过程和关键代码展示，对超参数的实验分析，最优超参数下的训练集、验证集损失曲线以及测试集上的实验结果。

Experiment 1

Experimental Requirements

Implement a feedforward neural network using PyTorch or TensorFlow to approximate the function:

$$y = \sin(x), \qquad x \in [0, 2\pi).$$

Study the impact of network depth, learning rate, network width, and activation function on model performance.

Experimental Steps

1. **Network Framework:** Choose either PyTorch or TensorFlow, and follow the instructions on the official website to install the package. If you need to use a GPU, you may also need to install CUDA drivers. This experiment can be completed using only the CPU, but it is still strongly recommended that everyone install the GPU version to meet the needs of future experiments.
2. **Data Generation:** The dataset for this experiment only needs to be automatically generated by the program: randomly sample x values within $[0, 2\pi)$ and calculate $y = \sin(x)$ as the corresponding y values. Generate three **disjoint** datasets as the training set, validation set, and test set. Training may only be conducted on the training set, and experimental parameter tuning may only be conducted on the validation set.
3. **Model Building:** Use modules provided by PyTorch or TensorFlow to construct the model, such as `torch.nn.Linear()` and `torch.nn.ReLU()`. You do not need to implement the underlying forward and backward processes manually.
4. **Model Training:** Input the generated training set into the model, calculate the forward loss, and perform backward gradient propagation to train the model. It is also recommended to use an optimizer provided by the framework to update the parameters. Record the loss on the training and validation sets during training and visualize it by plotting.
5. **Parameter Tuning Analysis:** Evaluate the trained model on the validation set using Mean Square Error (MSE) as the performance metric. Adjust hyperparameters such as network depth, learning rate, network width, and activation function; then retrain, evaluate, and analyze their impact on model performance.
6. **Testing Performance:** Select the set of hyperparameters that you consider most suitable (e.g., those that perform best on the validation set), retrain the model, and evaluate it on the test set. **This should be the only evaluation on the test set** in your experiment. Record the test result (MSE).

Experimental Submission

The deadline for this experiment is **November 14th, 23:59:59**.

Submit your work by email to liangyunfei@mail.ustc.edu.cn. The specific requirements are as follows:

1. Package all files into a compressed archive named `Student ID Name exp1.zip`.
2. The code should contain only `.py` files; do not include intermediate experimental results (such as saved datasets). If there are multiple files, place them in the `src/` folder.
3. Provide a `main.py` file that can run directly and output results, including saved training-loss and validation-loss curves as functions of epoch, and the MSE on the test set.
4. Provide a `requirements.txt` file listing all required dependent packages. Manually list all non-standard libraries and versions used in the code, or generate the file with `pip freeze > requirements.txt`.
5. The experimental report must be in PDF format and include the student's name and student ID. It should contain a brief experimental process and key code presentation, experimental analysis of hyperparameters, the training and validation loss curves under the optimal hyperparameters, and the experimental results on the test set.